Hiperparameters setting - Deflected Subgradient

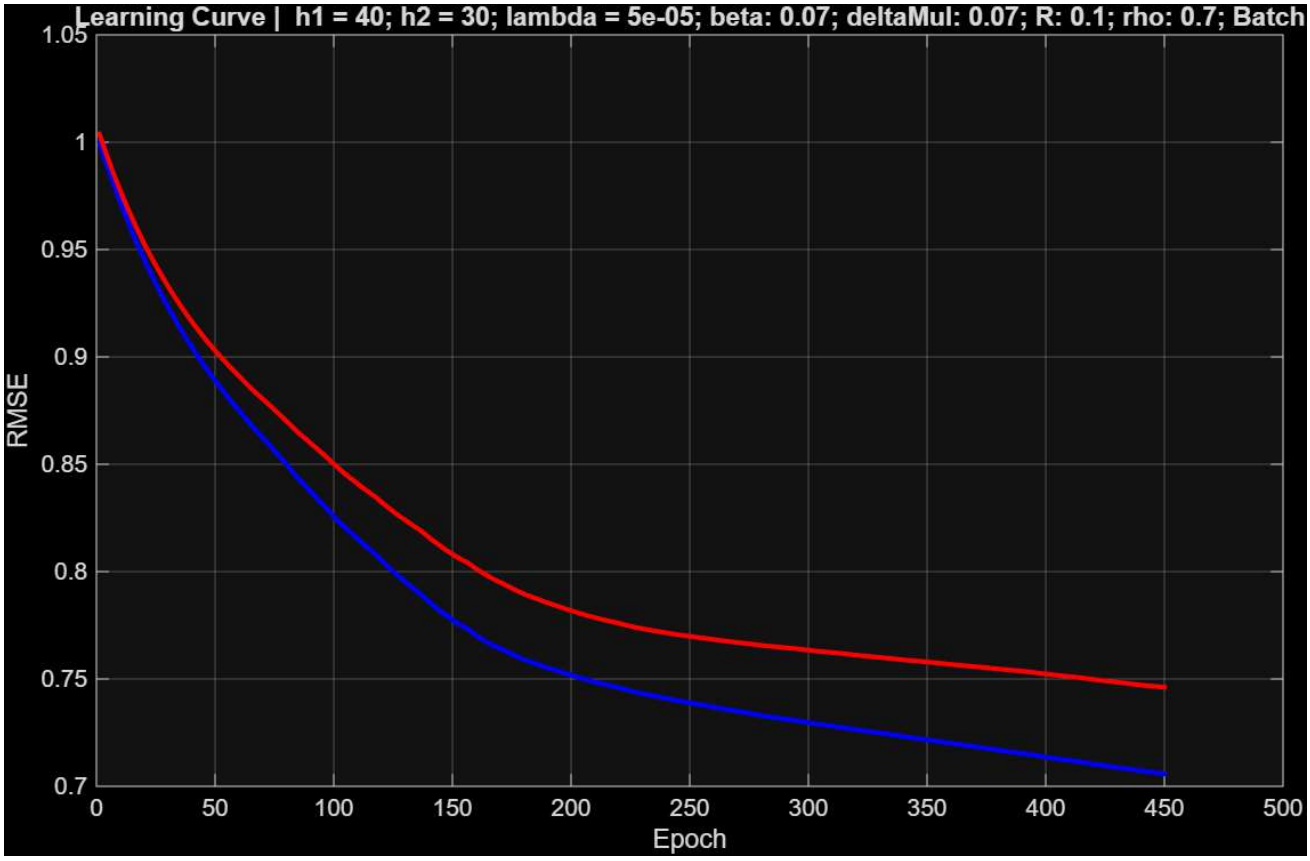
Final Model

Learning Algorithm	#Units, lambda, beta, delta, R, rho	RMSE(TR/VL/TS)
Stepsize-restricted	h1=30, h2=20, λ=1e-4, β=0.3, δ=0.1, R=0.1, ρ=0.7	0.5415/0.6737/0.7087

Some results abouts hyperparameters for Deflected Subgradient

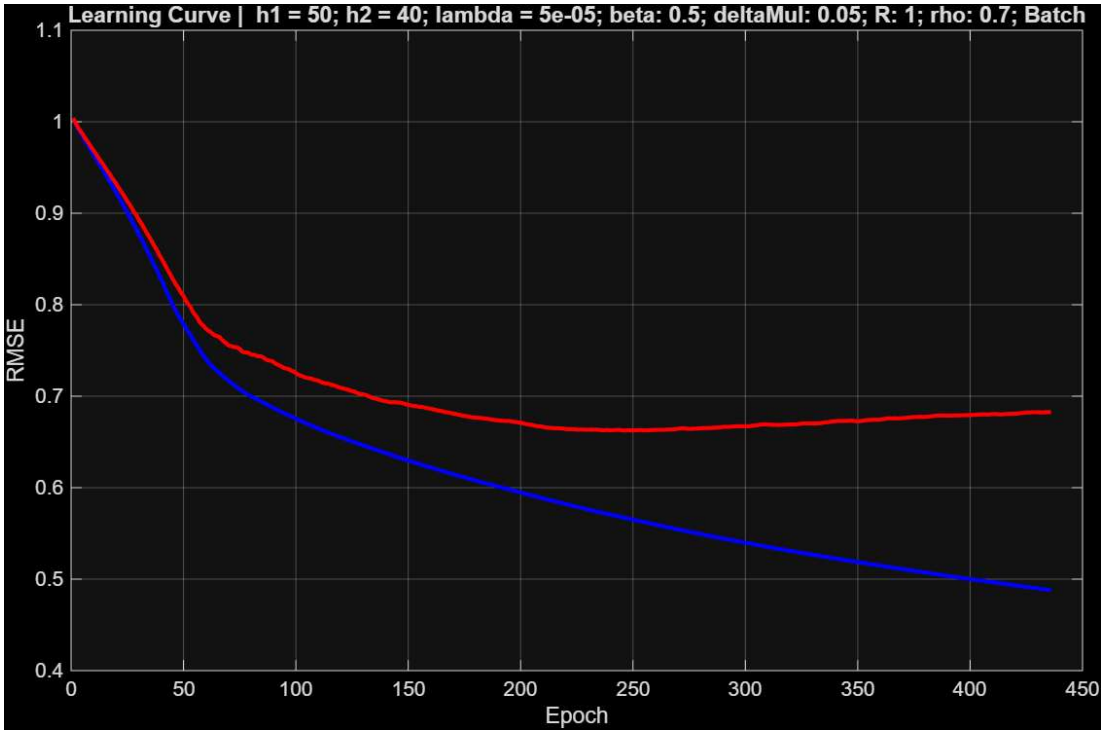
Red curve: Validation **Blue curve: Training**

h1=40, h2=30, λ=5e-5, β=0.07, δ=0.07, R=0.1, ρ=0.7



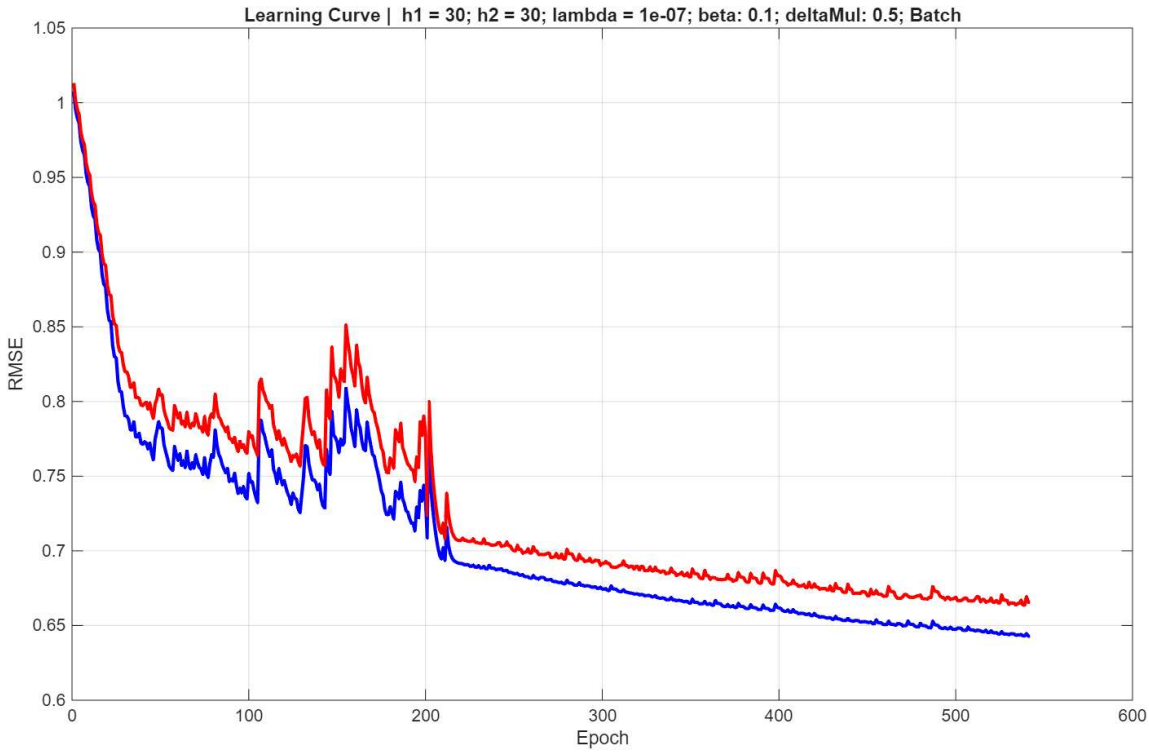
Lower values of **β** and **δ** were tested compared to the final model. As observed from both the training and validation RMSE, the model performs significantly worse (approximately 5 percentage points lower), due to a too slow descent, which triggers early stopping before full convergence. This behavior was observed in the majority of models tested with **β** and **δ** values lower than those of the final model.

h1=50, h2=40, λ=5e-5, β=0.5, δ=0.05, R=1, ρ=0.7



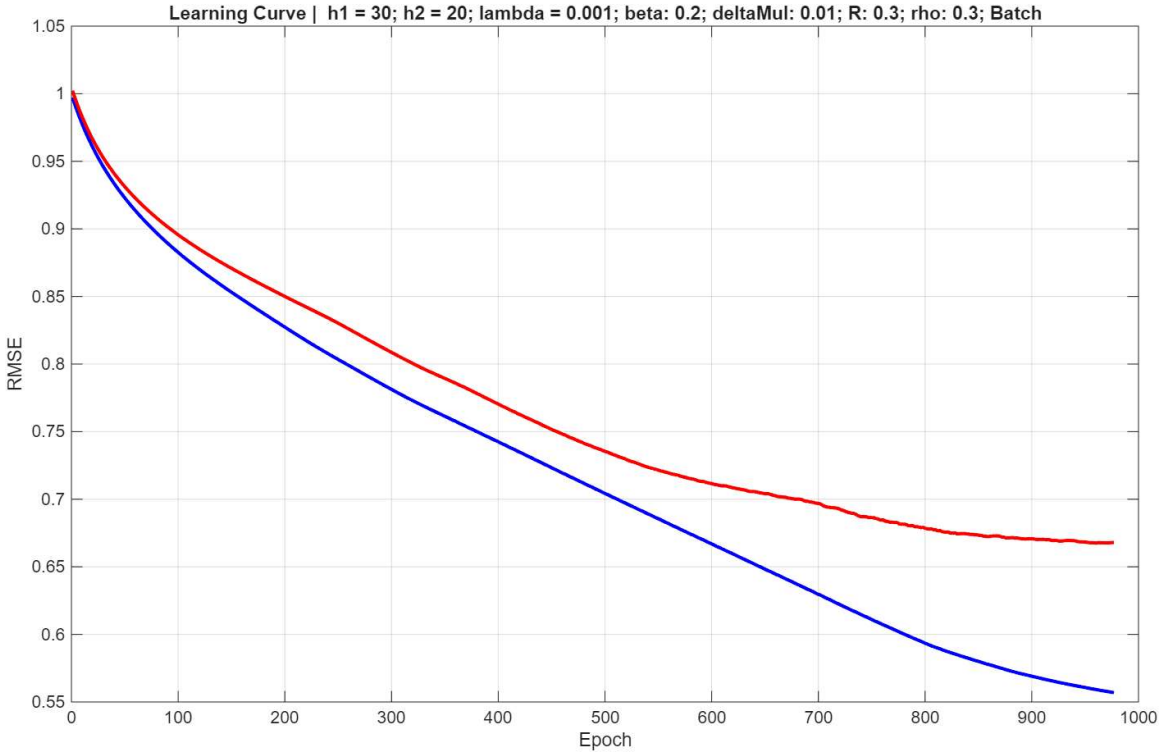
In this case, however, higher values of the hyperparameters **R** and **β** were tested in an attempt to speed up convergence. As seen from the validation curve, these overly aggressive values lead to overfitting.

h1=30, h2=30, λ=1e-7, β=0.1, δ=0.5, R=1, ρ=0.7



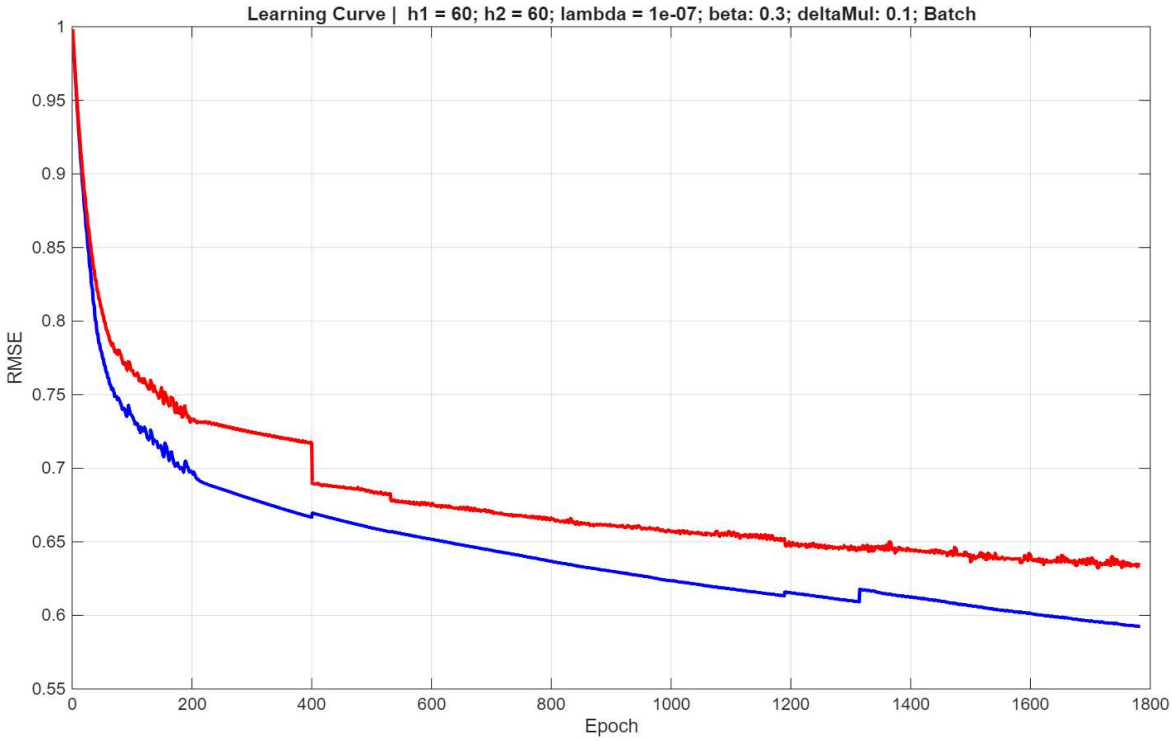
These unsmooth curves are caused by a too low **λ** value combined with an overly aggressive **δ** in the *f** update.

h1=30, h2=20, λ=1e-3, β=0.2, δ=0.01, R=0.3, ρ=0.3



In this model, we tested lower values of **δ** and **ρ** to reduce the step size and potentially improve validation performance. However, the learning curve again exhibited slow convergence, without any noticeable improvement in results.

h1=60, h2=60, λ=1e-7, β=0.3, δ=0.1, R=0.1, ρ=0.7



Finally, by increasing the number of neurons while keeping the other hyperparameters at the levels of the best model, we observed irregular behavior:

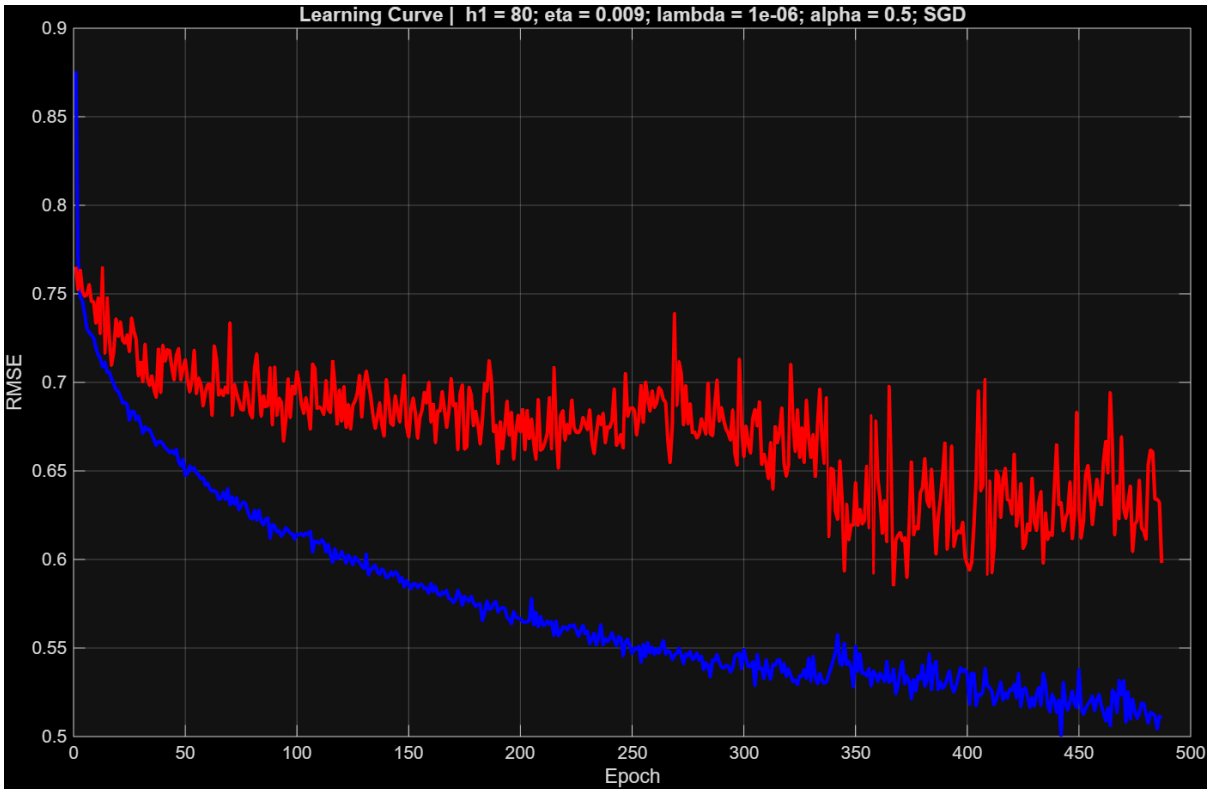
some folds terminated very early, while others reached slightly better convergence than the final model, but with irregular, unsmooth learning curves.

HYPERPARAMETER SETTING – CLASSIC NN

Training Time Comparison

Below there are two plots showing the improvement of training time between the initial implementation of SGD (with OOP constructs) and the implementation of mini-batch used for grid search, which exploits the efficiency of MATLAB computations with matrixes and vectors.

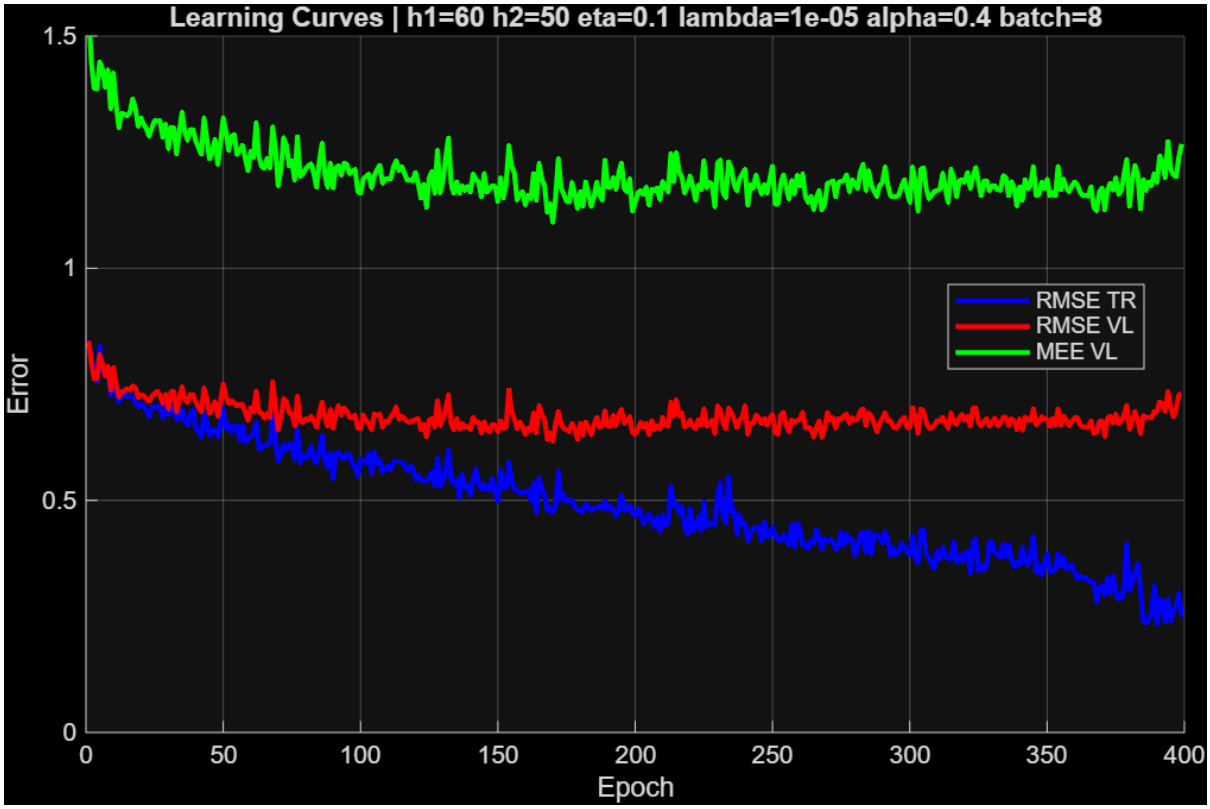
SGD



rmse_training_min (norm.) = 0.4971
rmse_validation_min (norm.) = 0.5835
mee_training_min (true scale) = 22.0860

For 5 folds:
<u>training_time = 6.831e+03 seconds (roughly 2 hours)</u>

Mini-batch (pseudo-SGD)



rmse_training_min (norm.) = 0.2341
rmse_validation_min (norm.) = 0.6278
mee_validation_mean (true scale) = 22.7992

For 5 folds:
<u>training_time = 22.7887 seconds</u>

The reason behind this improvement lies in the fact that:

- With the OOP implementation of SGD, we are managing tens of objects representing neurons, each of which has its own state and properties to maintain in memory.
  Accessing them requires lots of time and storing them requires lots of space.

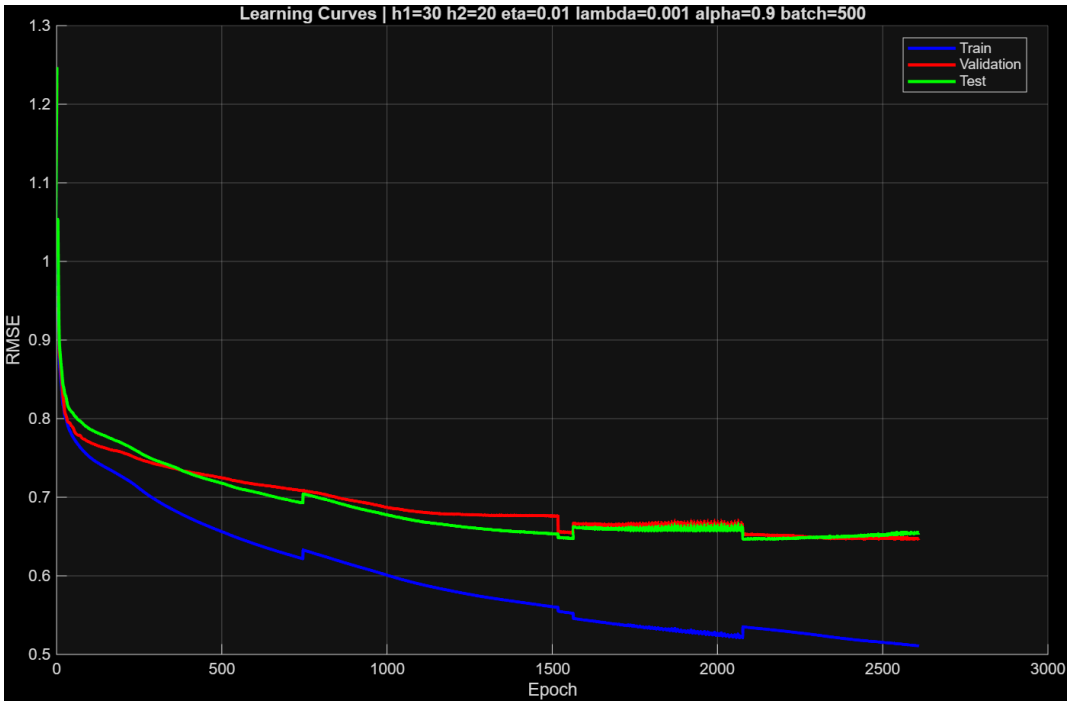
- Meanwhile, keeping just the weights and biases in a few vectors improves these aspects dramatically.

Although it could have been obvious, the improvement was so huge that it had to be mentioned.

Alpha hyperparameter (Heavy-Ball Momentum)

Alpha = 0

Alpha = 0.9



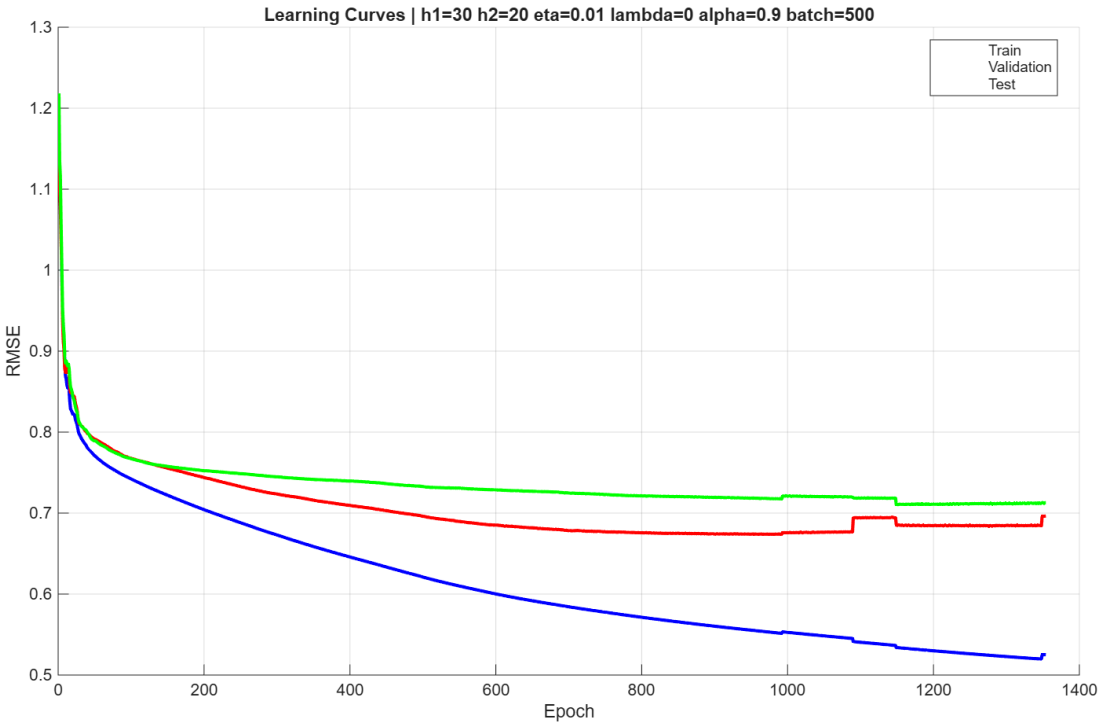
As we can observe:

- Without momentum, the learning is stable but conservative, so it converges at a much slower rate and plateaus earlier, requiring many epochs to reach comparable performance.

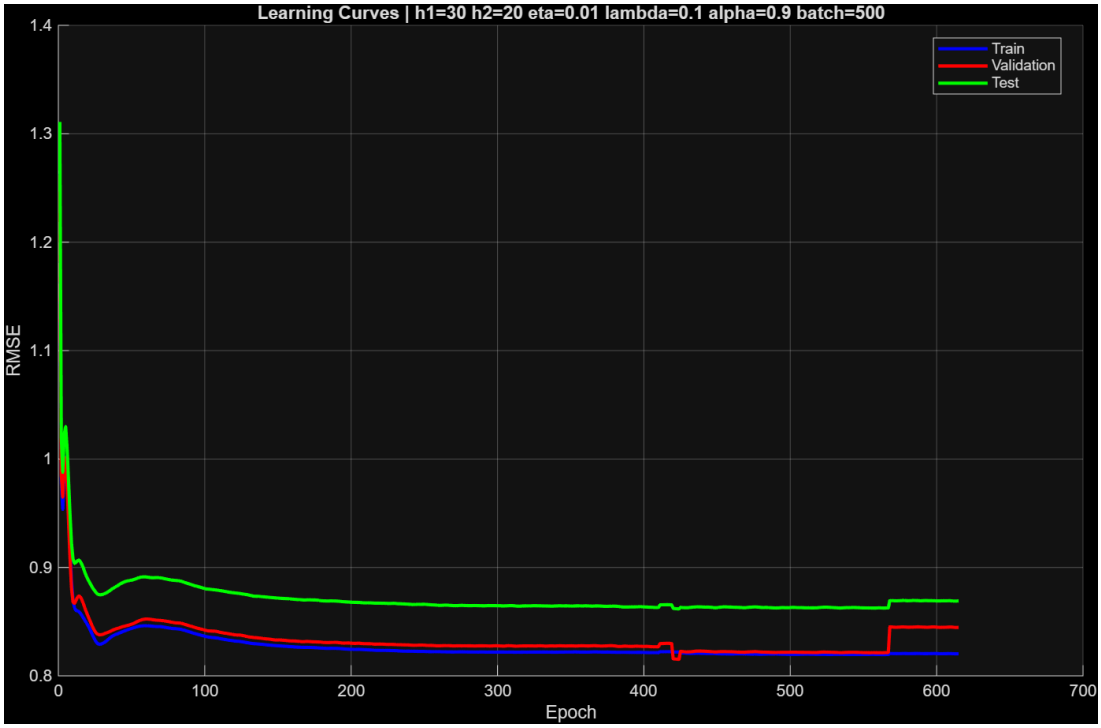
- With momentum, we introduce inertia, which accelerates the descent to avoid plateaus and thus improves both convergence speed and final performances, at the cost of very mild oscillations (epoch 1600-2000).

Lambda hyperparameter (L1 Regularization)

Lambda = 0



Lambda = 0.1



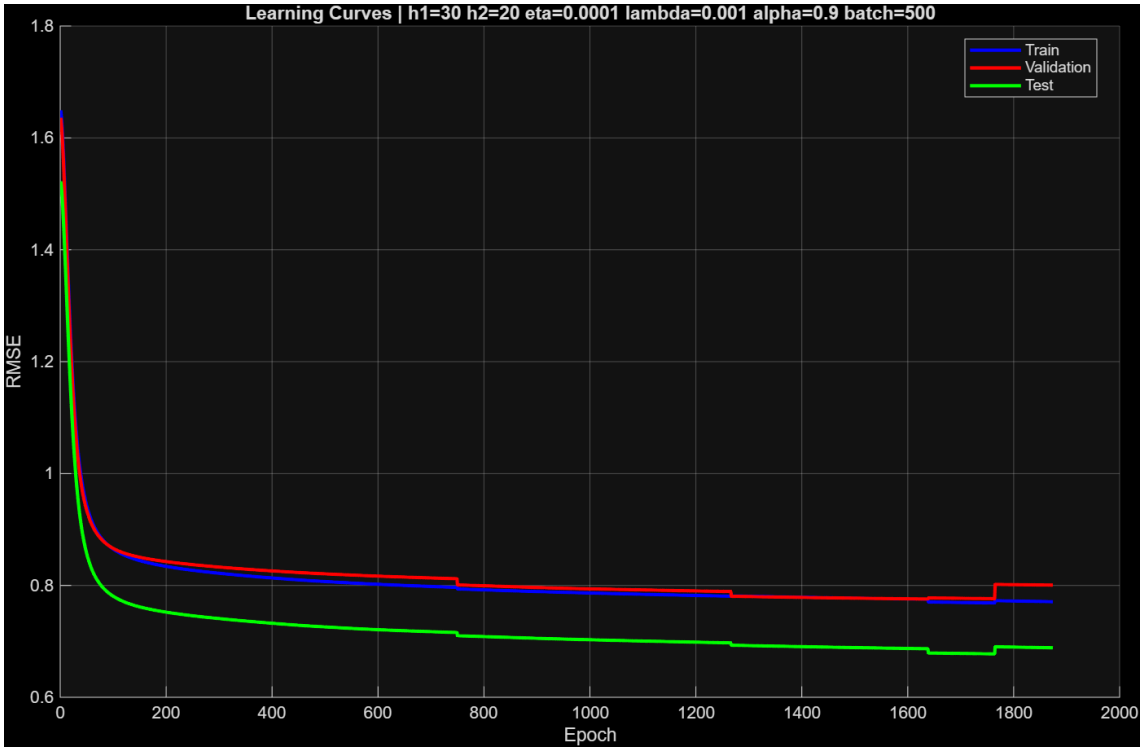
As we can observe:

- Without regularization, the model still has high capacity and can fit the training data, hence the low training error, but this does not translate to a better generalization, since the gap between the two curves is huge. A clear case of overfitting.

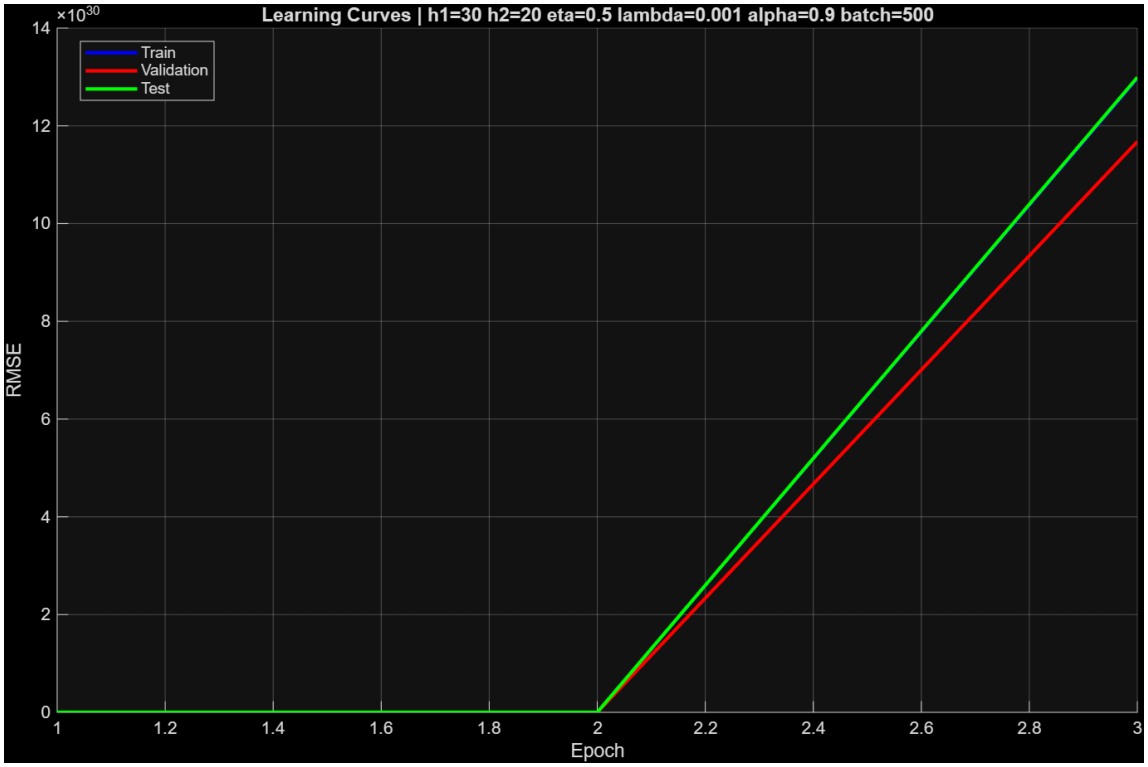
- With very high regularization, we basically nullify the gradient computation effect and all the other factors, thus the curves plateau very early to a fixed value and nothing will move them from there. This is a case of underfitting.

Eta hyperparameter (Learning Rate)

Eta = 0.0001



Eta = 0.5



As we can observe:

- With very low learning rate, the model is not learning much from the training data, hence the very high training error. In fact, it is learning so little from it that even the test data is more predictable than the actual data it is training on.

- With very high learning rate, even the slightest noise in data can be amplified and disrupt the entire training, making it never converge.